

CO3 AT1 ANSWERS

BY ANTONY MAGRIN(192572103)

Q1. For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys. Given Relation

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

Where:

- **SID** – Student ID
- **SName** – Student Name
- **CID** – Course ID
- **CName** – Course Name
- **Instructor** – Course Instructor
- **Grade** – Grade obtained by the student

Step 1: Identify the Functional Dependencies (FDs)

From the given relation, the following functional dependencies can be identified:

1. **SID $\rightarrow$ SName**
 - a. A Student ID uniquely identifies a student's name.
2. **CID $\rightarrow$ CName, Instructor**
 - a. A Course ID uniquely determines the course name and instructor.
3. **SID, CID $\rightarrow$ Grade**
 - a. A student's grade depends on both the student and the course.

Step 2: List of Functional Dependencies

SIDCIDSID, CID $\rightarrow$ SName $\rightarrow$ CName, Instructor $\rightarrow$ Grade

Step 3: Determine the Candidate Key

To identify every attribute in the relation, we compute the closure of {SID, CID}.

Closure of (SID, CID)

Using the functional dependencies:

- $SID \rightarrow SName$
- $CID \rightarrow CName, Instructor$
- $SID, CID \rightarrow Grade$

Therefore,

$(SID, CID)^+ = \{SID, SName, CID, CName, Instructor, Grade\}$

Since all attributes are obtained, (SID, CID) is a **Super Key**.

Neither **SID** nor **CID** alone can determine all the attributes, so (SID, CID) is the **minimal Super Key**, i.e., the **Candidate Key**.

Step 4: Candidate Key

Candidate Key = $\{SID, CID\}$

Final Answer

Functional Dependencies

- $SID \rightarrow SName$
- $CID \rightarrow CName, Instructor$
- $SID, CID \rightarrow Grade$

Candidate Key

$\{SID, CID\}$

Conclusion

The relation **STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)** has three functional dependencies. The combination (SID, CID) uniquely identifies every tuple in the relation and is therefore the **Candidate Key**. This key ensures that each student's grade is uniquely associated with a specific course while avoiding ambiguity.

Q2. Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

Answer

Definition of First Normal Form (1NF)

First Normal Form (1NF) states that every attribute in a relation must contain **only atomic (single) values**, and there should be **no repeating groups or multi-valued attributes**.

Step 1: Unnormalized Relation (UNF)

Consider the following relation:

STUDENT

SID	Student Name	Courses
S101	Rahul	DBMS, Java, Python
S102	Priya	C++, DBMS
S103	Arjun	Java

Observation

The **Courses** attribute contains multiple values in a single cell.

Hence, the relation is **not in First Normal Form (1NF)**.

Step 2: Identify the Problem

The relation violates 1NF because:

- The **Courses** attribute is a **multi-valued attribute**.
- Each field should contain only **one value**.

Step 3: Convert into First Normal Form (1NF)

Separate each course into a different row.

STUDENT (1NF)

SID	Student Name	Course
S101	Rahul	DBMS
S101	Rahul	Java
S101	Rahul	Python
S102	Priya	C++
S102	Priya	DBMS
S103	Arjun	Java

Now,

- Each cell contains **only one value**.
- There are **no repeating groups**.
- Every record is unique.

Therefore, the relation satisfies **First Normal Form (1NF)**.

Step 4: Justification

The conversion to **1NF** is successful because:

- Multi-valued attributes are removed.
- Each row stores only one course for one student.
- Every attribute contains an atomic value.
- Data becomes easier to search, update, and maintain.

Advantages of 1NF

- Eliminates repeating groups.
- Improves data consistency.
- Simplifies searching and updating records.
- Forms the foundation for higher normal forms (2NF, 3NF, BCNF).

Final Answer

Unnormalized Relation (UNF)

SID	Student Name	Courses
S101	Rahul	DBMS, Java, Python
S102	Priya	C++, DBMS
S103	Arjun	Java

First Normal Form (1NF)

SID	Student Name	Course
S101	Rahul	DBMS
S101	Rahul	Java
S101	Rahul	Python
S102	Priya	C++
S102	Priya	DBMS
S103	Arjun	Java

Conclusion

The given unnormalized relation was converted into **First Normal Form (1NF)** by replacing the multi-valued **Courses** attribute with atomic values. Each row now contains a single course for a student, satisfying the rules of **1NF** and reducing data redundancy.

Q3. Given relation R(A, B, C, D, E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.

Answer

Given

Relation:

R(A, B, C, D, E)

Functional Dependencies (FDs):

- $A \rightarrow BC$
- $BC \rightarrow D$

- $D \rightarrow E$

Step 1: Find the Candidate Key

Find the closure of **A**.

$$A^+ = \{A\}$$

Using the given FDs:

- $A \rightarrow BC \Rightarrow \{A, B, C\}$
- $BC \rightarrow D \Rightarrow \{A, B, C, D\}$
- $D \rightarrow E \Rightarrow \{A, B, C, D, E\}$

Therefore,

$$A^+ = \{A, B, C, D, E\}$$

Hence,

Candidate Key = A

Step 2: Check for Partial Dependency

A relation is **not in 2NF** if:

- It has a **composite candidate key**, and
- A non-prime attribute depends on only part of that key.

In this relation:

- Candidate Key = **A**
- It is a **single attribute key** (not composite).

Therefore,

there are no partial dependencies.

Step 3: Verify 2NF

Since:

- The relation is already in **1NF**.
- The candidate key consists of only one attribute.
- No non-prime attribute depends on a part of the key.

Hence,

the relation already satisfies Second Normal Form (2NF).

Step 4: 2NF Decomposition

Since there are **no partial dependencies, no decomposition is required.**

The relation remains:

R(A, B, C, D, E)

Final Answer

Candidate Key

A

Functional Dependencies

- $A \rightarrow BC$
- $BC \rightarrow D$
- $D \rightarrow E$

Partial Dependency

No partial dependency exists because the candidate key is a single attribute (A).

2NF Decomposition

No decomposition is required.

The relation is already in **Second Normal Form (2NF)**.

Conclusion

The closure of **A** includes all attributes of the relation, making **A** the candidate key. Since the candidate key is a **single attribute**, partial dependencies cannot occur. Therefore, the relation already satisfies **Second Normal Form (2NF)**, and no decomposition is necessary.

Q4. Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

Answer

Given Relation

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)

Step 1: Identify the Functional Dependencies

The functional dependencies are:

1. **EmpID $\rightarrow$ EmpName, DeptID**
2. **DeptID $\rightarrow$ DeptName, ManagerID**

From these dependencies,

EmpID $\rightarrow$ DeptID $\rightarrow$ (DeptName, ManagerID)

This indicates the presence of a **transitive dependency**.

Step 2: Identify the Candidate Key

The attribute **EmpID** uniquely identifies each employee.

Therefore,

Candidate Key = EmpID

Step 3: Identify the Transitive Dependency

Since:

- **EmpID** → **DeptID**
- **DeptID** → **DeptName, ManagerID**

Therefore,

EmpID → **DeptName, ManagerID** (through DeptID)

This is a **transitive dependency** because non-key attributes (**DeptName** and **ManagerID**) depend on another non-key attribute (**DeptID**) rather than directly on the primary key.

Step 4: Decompose the Relation into 3NF

To remove the transitive dependency, split the relation into two tables.

Relation 1: EMPLOYEE

EmpID	EmpName	DeptID
E101	Rahul	D01
E102	Priya	D02
E103	Arjun	D01

Primary Key: EmpID

Relation 2: DEPARTMENT

DeptID	DeptName	ManagerID
D01	CSE	M101

D02 IT M102

Primary Key: DeptID

Step 5: Verify 3NF

EMPLOYEE Table

- **EmpID $\rightarrow$ EmpName, DeptID**
- All non-key attributes depend directly on the primary key.
- No transitive dependency exists.

DEPARTMENT Table

- **DeptID $\rightarrow$ DeptName, ManagerID**
- All non-key attributes depend only on the primary key.

Hence, both relations satisfy **Third Normal Form (3NF)**.

Final Answer

Candidate Key

EmpID

Transitive Dependency

EmpID $\rightarrow$ DeptID $\rightarrow$ (DeptName, ManagerID)

3NF Decomposition

EMPLOYEE(EmpID, EmpName, DeptID)

EmpID	EmpName	DeptID
E101	Rahul	D01
E102	Priya	D02
E103	Arjun	D01

DEPARTMENT(DeptID, DeptName, ManagerID)

DeptID	DeptName	ManagerID
D01	CSE	M101
D02	IT	M102

Conclusion

The original **EMPLOYEE** relation contained a **transitive dependency** because **DeptName** and **ManagerID** depended on **DeptID**, which in turn depended on **EmpID**. By decomposing the relation into **EMPLOYEE** and **DEPARTMENT**, the transitive dependency is eliminated, redundancy is reduced, and both relations satisfy **Third Normal Form (3NF)**.

Q5. Verify whether R(A, B, C, D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.

Answer

Given

Relation:

R(A, B, C, D)

Functional Dependencies (FDs):

- $AB \rightarrow C$
- $C \rightarrow D$
- $D \rightarrow A$

Step 1: Find the Candidate Key

Find the closure of **AB**.

(AB)⁺

Using the given FDs:

- $AB \rightarrow C \Rightarrow \{A, B, C\}$

- $C \rightarrow D \Rightarrow \{A, B, C, D\}$
- $D \rightarrow A$ (already present)

Therefore,

$$(AB)^+ = \{A, B, C, D\}$$

Hence,

Candidate Key = AB

Step 2: Check BCNF Condition

A relation is in **Boyce-Codd Normal Form (BCNF)** if the determinant of every functional dependency is a super key.

Check each FD:

1. $AB \rightarrow C$

- Determinant = **AB**
- **AB** is the candidate key.

Satisfies BCNF

2. $C \rightarrow D$

- Determinant = **C**
- **C** is **not** a candidate key.

Violates BCNF

3. $D \rightarrow A$

- Determinant = **D**
- **D** is **not** a candidate key.

Violates BCNF

Step 3: Decompose into BCNF

Since $C \rightarrow D$ violates BCNF, decompose the relation into:

Relation $R_1(C, D)$

Functional Dependency:

$$C \rightarrow D$$

Primary Key:

C

This relation is in **BCNF** because **C** is the key.

Relation $R_2(A, B, C)$

Functional Dependency:

$$AB \rightarrow C$$

Primary Key:

AB

This relation is also in **BCNF** because **AB** is the candidate key.

Step 4: Verify the Decomposed Relations

$R_1(C, D)$

- FD: $C \rightarrow D$
- Candidate Key: **C**
- **BCNF Satisfied**

R₂(A, B, C)

- FD: **AB** → C
- Candidate Key: **AB**
- **BCNF Satisfied**

Thus, both relations are in **BCNF**.

Final Answer

Candidate Key

AB

BCNF Verification

Functional Dependency	BCNF Status
AB → C	Satisfies BCNF
C → D	Violates BCNF
D → A	Violates BCNF

BCNF Decomposition

R₁(C, D)

C	D
C1	D1
C2	D2

Primary Key: C

R₂(A, B, C)

A	B	C
A1	B1	C1
A2	B2	C2

Primary Key: AB

Conclusion

The relation $R(A, B, C, D)$ is **not in BCNF** because the functional dependencies $C \rightarrow D$ and $D \rightarrow A$ have determinants that are **not super keys**. By decomposing the relation into $R_1(C, D)$ and $R_2(A, B, C)$, all functional dependencies satisfy the BCNF condition, thereby eliminating redundancy and improving data integrity.

Q6. Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

Answer

Definition

A **Lossless-Join Decomposition** ensures that after decomposing a relation into smaller relations, the original relation can be reconstructed exactly by performing a natural join without losing any information.

The **Chase Algorithm (Tabular Method)** is used to verify whether a decomposition is **lossless** or **lossy**.

Given Relation

Consider the relation:

$R(A, B, C)$

Functional Dependency:

$A \rightarrow B$

Decompose the relation into:

- $R_1(A, B)$
- $R_2(A, C)$

Verify whether the decomposition is **lossless** using the Chase Algorithm.

Step 1: Construct the Initial Chase Table

Relation	A	B	C
$R_1(A, B)$	a	b	c_1
$R_2(A, C)$	a	b_2	c

Here,

- Common attribute **A** is represented by the same symbol **a**.
- Different attributes are represented by different symbols.

Step 2: Apply the Functional Dependency

Functional Dependency:

$A \rightarrow B$

Since both rows have the same value of **A** (**a**), the value of **B** must also be the same.

Replace **b₂** with **b**.

Updated table:

Relation	A	B	C
$R_1(A, B)$	a	b	c_1
$R_2(A, C)$	a	b	c

Step 3: Verify Lossless Join

After applying the functional dependency:

- The rows contain the same values for the common attribute **A**.
- The dependency $A \rightarrow B$ has been satisfied.
- The original relation can be reconstructed exactly by joining **R₁** and **R₂**.

Therefore, the decomposition is **Lossless**.

Final Answer

Original Relation

$R(A, B, C)$

Functional Dependency

$A \rightarrow B$

Decomposition

- $R_1(A, B)$
- $R_2(A, C)$

Chase Table

Relation	A	B	C
$R_1(A, B)$	a	b	c ₁
$R_2(A, C)$	a	b	c

Result

Since the common attribute **A** determines **B**, the decomposition satisfies the **lossless-join property**.

Conclusion

Using the **Chase (Tabular) Algorithm**, the decomposition of $R(A, B, C)$ into $R_1(A, B)$ and $R_2(A, C)$ is verified to be **lossless**. This means the original relation can be reconstructed through a natural join without any loss of information or creation of spurious tuples.

Q7. Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

Answer

Definition of Fourth Normal Form (4NF)

A relation is said to be in **Fourth Normal Form (4NF)** if it is in **BCNF** and contains **no non-trivial multivalued dependencies (MVDs)**.

A **Multivalued Dependency (MVD)** exists when one attribute independently determines multiple values of another attribute.

Given Relation

TEACHER(TID, Subject, Hobby)

Where:

- **TID** – Teacher ID
- **Subject** – Subject taught
- **Hobby** – Teacher's Hobby

Step 1: Identify the Multi-Valued Dependencies

Assume a teacher can teach **multiple subjects** and also have **multiple hobbies**, where subjects and hobbies are independent of each other.

The multivalued dependencies are:

- **TID $\twoheadrightarrow$ Subject**
- **TID $\twoheadrightarrow$ Hobby**

These indicate that one teacher may be associated with many subjects and many hobbies independently.

Step 2: Example of the Relation

TEACHER

TID	Subject	Hobby
T101	DBMS	Reading
T101	DBMS	Music
T101	Java	Reading
T101	Java	Music

Here, every subject is combined with every hobby, resulting in **redundant tuples**.

Step 3: Decompose into 4NF

To eliminate the multivalued dependencies, decompose the relation into two separate relations.

Relation 1: TEACHER_SUBJECT

TID	Subject
-----	---------

T101	DBMS
------	------

T101	Java
------	------

Primary Key: (TID, Subject)

Relation 2: TEACHER_HOBBY

TID	Hobby
-----	-------

T101	Reading
------	---------

T101	Music
------	-------

Primary Key: (TID, Hobby)

Step 4: Verify 4NF

- In **TEACHER_SUBJECT**, each teacher is associated only with the subjects they teach.
- In **TEACHER_HOBBY**, each teacher is associated only with their hobbies.
- The multivalued dependencies are removed.
- Both relations satisfy **Fourth Normal Form (4NF)**.

Final Answer

Multivalued Dependencies

- **TID $\twoheadrightarrow$ Subject**
- **TID $\twoheadrightarrow$ Hobby**

4NF Decomposition

TEACHER_SUBJECT(TID, Subject)

TID	Subject
T101	DBMS
T101	Java

TEACHER_HOBBY(TID, Hobby)

TID	Hobby
T101	Reading
T101	Music

Conclusion

The relation **TEACHER(TID, Subject, Hobby)** contains the multivalued dependencies **TID** $\twoheadrightarrow$ **Subject** and **TID** $\twoheadrightarrow$ **Hobby**, which lead to redundancy. By decomposing it into **TEACHER_SUBJECT(TID, Subject)** and **TEACHER_HOBBY(TID, Hobby)**, the multivalued dependencies are eliminated, redundancy is reduced, and the relations satisfy **Fourth Normal Form (4NF)**.

Q8. Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

Answer

Definition of Fifth Normal Form (5NF)

Fifth Normal Form (5NF), also known as **Project-Join Normal Form (PJNF)**, ensures that a relation cannot be decomposed further without losing information. It eliminates **join dependencies** and prevents redundancy caused by unnecessary combinations of data.

Step 1: Consider the Relation

SPJ(Supplier, Part, Project)

Where:

- **Supplier** – Supplier ID
- **Part** – Part ID
- **Project** – Project ID

Sample Relation

Supplier	Part	Project
S1	P1	J1
S1	P2	J1
S2	P1	J1
S2	P2	J2

Step 2: Join Dependency

Suppose:

- A supplier supplies many parts.
- A supplier works on many projects.
- A part is used in many projects.

When all three attributes are stored in a single relation, **redundant tuples** may be created due to repeated combinations.

This redundancy is caused by a **Join Dependency (JD)**.

Step 3: Decompose into 5NF

To eliminate redundancy, decompose the relation into three smaller relations.

Relation 1: SUPPLIER_PART

Supplier	Part
S1	P1
S1	P2
S2	P1
S2	P2

Relation 2: SUPPLIER_PROJECT

Supplier	Project
S1	J1
S2	J1
S2	J2

Relation 3: PART_PROJECT

Part	Project
P1	J1
P2	J1
P2	J2

Step 4: Verify 5NF

- Each relation stores information about only one relationship.
- Redundant combinations are eliminated.
- The original relation can be reconstructed by joining the three relations without any loss of information.
- Therefore, the decomposition satisfies **Fifth Normal Form (5NF)**.

Final Answer

Join Dependency

The relation **SPJ(Supplier, Part, Project)** contains a **join dependency**, leading to redundant combinations of suppliers, parts, and projects.

5NF Decomposition

SUPPLIER_PART(Supplier, Part)

Supplier	Part
S1	P1
S1	P2
S2	P1
S2	P2

SUPPLIER_PROJECT(Supplier, Project)

Supplier	Project
S1	J1
S2	J1
S2	J2

PART_PROJECT(Part, Project)

Part	Project
P1	J1
P2	J1
P2	J2

Conclusion

The relation **SPJ(Supplier, Part, Project)** contains a **join dependency** that introduces redundancy. By decomposing it into **SUPPLIER_PART**, **SUPPLIER_PROJECT**, and **PART_PROJECT**, the redundancy is eliminated while preserving all information. The resulting relations satisfy **Fifth Normal Form (5NF)** because the original relation can be reconstructed through natural joins without any loss of data.

Q9. Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

Answer

Given Relation

Consider the following poorly designed relation:

COLLEGE(StudentID, StudentName, CourseID, CourseName, FacultyID, FacultyName)

Sample Relation

StudentID	StudentName	CourseID	CourseName	FacultyID	FacultyName
S101	Rahul	C101	DBMS	F01	Dr. Kumar
S102	Priya	C101	DBMS	F01	Dr. Kumar
S103	Arjun	C102	Java	F02	Dr. Meena

This relation contains redundant data because **CourseName** and **FacultyName** are repeated for every student enrolled in the same course.

Step 1: Identify the Anomalies

1. Insertion Anomaly

A new course cannot be added unless at least one student is enrolled in it.

Example: We cannot insert Course **C103 (Python)** unless a student registers for it.

2. Update Anomaly

If the faculty name changes, it must be updated in every row.

Example: If **Dr. Kumar** changes to **Dr. Raj Kumar**, all rows containing **Dr. Kumar** must be updated. Failure to update all rows leads to inconsistent data.

3. Deletion Anomaly

If the last student enrolled in a course is deleted, the course and faculty information are also lost.

Example: Deleting the record of **S103** removes all information about **Course C102 (Java)** and **Faculty F02**.

Step 2: Functional Dependencies

- **StudentID** → **StudentName**
- **CourseID** → **CourseName, FacultyID**
- **FacultyID** → **FacultyName**

Step 3: Normalize to Third Normal Form (3NF)

Decompose the relation into three relations.

1. STUDENT

StudentID	StudentName
S101	Rahul

S102	Priya
S103	Arjun

Primary Key: StudentID

2. COURSE

CourseID	CourseName	FacultyID
C101	DBMS	F01
C102	Java	F02

Primary Key: CourseID

3. FACULTY

FacultyID	FacultyName
F01	Dr. Kumar
F02	Dr. Meena

Primary Key: FacultyID

Step 4: Verify 3NF

- In the **STUDENT** table, all attributes depend only on **StudentID**.
- In the **COURSE** table, all attributes depend only on **CourseID**.
- In the **FACULTY** table, **FacultyName** depends only on **FacultyID**.
- There are **no partial or transitive dependencies**.

Hence, all three relations satisfy **Third Normal Form (3NF)**.

Final Answer

Anomalies Identified

- **Insertion Anomaly:** Cannot insert a new course without a student.
- **Update Anomaly:** Faculty details must be updated in multiple rows.

- **Deletion Anomaly:** Deleting the last student removes course and faculty details.

3NF Decomposition

STUDENT(StudentID, StudentName)

StudentID	StudentName
S101	Rahul
S102	Priya
S103	Arjun

COURSE(CourseID, CourseName, FacultyID)

CourseID	CourseName	FacultyID
C101	DBMS	F01
C102	Java	F02

FACULTY(FacultyID, FacultyName)

FacultyID	FacultyName
F01	Dr. Kumar
F02	Dr. Meena

Conclusion

The original college database contained **insertion, update, and deletion anomalies** due to redundant data. By decomposing it into the **STUDENT**, **COURSE**, and **FACULTY** relations, redundancy is eliminated, data consistency is improved, and the database satisfies **Third Normal Form (3NF)**.

Q10. Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

Answer

Given Relation

Consider the relation:

R(A, B, C, D, E)

Functional Dependencies (FDs):

- $A \rightarrow BC$
- $BC \rightarrow D$
- $D \rightarrow E$

Step 1: Compute the Attribute Closure

Closure of A (A^+)

Initially,

$$A^+ = \{A\}$$

Apply the functional dependencies one by one:

- $A \rightarrow BC$

$$A^+ = \{A, B, C\}$$

- $BC \rightarrow D$

$$A^+ = \{A, B, C, D\}$$

- $D \rightarrow E$

$$A^+ = \{A, B, C, D, E\}$$

Therefore,

$$A^+ = \{A, B, C, D, E\}$$

Since the closure contains **all attributes**, **A is a Candidate Key**.

Step 2: Compute Closures of Other Attributes

B^+

$$B^+ = \{B\}$$

No functional dependency can be applied.

Hence,

$$\mathbf{B}^+ = \{\mathbf{B}\}$$

$$\mathbf{C}^+$$

$$\mathbf{C}^+ = \{\mathbf{C}\}$$

No functional dependency applies.

Hence,

$$\mathbf{C}^+ = \{\mathbf{C}\}$$

$$\mathbf{D}^+$$

$$\mathbf{D}^+ = \{\mathbf{D}\}$$

Using $\mathbf{D} \rightarrow \mathbf{E}$

$$\mathbf{D}^+ = \{\mathbf{D}, \mathbf{E}\}$$

Hence,

$$\mathbf{D}^+ = \{\mathbf{D}, \mathbf{E}\}$$

$$\mathbf{E}^+$$

$$\mathbf{E}^+ = \{\mathbf{E}\}$$

No dependency applies.

Hence,

$$\mathbf{E}^+ = \{\mathbf{E}\}$$

Step 3: Determine Candidate Keys

A **Candidate Key** is the smallest set of attributes whose closure contains all attributes of the relation.

Since

$$A^+ = \{A, B, C, D, E\}$$

and no proper subset of **A** exists (it is a single attribute),

the only **Candidate Key** is:

$$\{A\}$$

Final Answer

Functional Dependencies

- $A \rightarrow BC$
- $BC \rightarrow D$
- $D \rightarrow E$

Attribute Closures

Attribute Set	Closure
A^+	$\{A, B, C, D, E\}$
B^+	$\{B\}$
C^+	$\{C\}$
D^+	$\{D, E\}$
E^+	$\{E\}$

Candidate Key

$$\{A\}$$

Conclusion

The closure of **A** includes all the attributes in the relation, i.e., $A^+ = \{A, B, C, D, E\}$. Therefore, **A** is the **only Candidate Key**. The closures of **B**, **C**, **D**, and **E** do not contain all attributes, so they cannot be considered candidate keys.